Machine Translated by Google

CAPÍTULO 4

# Hilos y Concurrencia



## Ceremonias

4.8 Proporcione dos ejemplos de programación en los que el uso de subprocesos múltiples no ofrezca un mejor rendimiento que una solución de un solo subproceso.

4.9 ¿En qué circunstancias una solución multihilo que utiliza múltiples hilos del kernel proporciona un mejor rendimiento que una solución de un solo hilo en un sistema de un solo procesador?

4.10 ¿Cuáles de los siguientes componentes del estado del programa se comparten entre ¿Hilos en un proceso multihilo?

a. Valores de registro

b. Memoria de montón

c. Variables globales

d. Memoria de pila

4.11 ¿Puede una solución multihilo que utiliza múltiples hilos a nivel de usuario lograr un mejor rendimiento en un sistema multiprocesador que en un sistema de un solo procesador? Explique.

4.12 En el capítulo 3, analizamos el navegador Chrome de Google y su práctica de abrir cada pestaña nueva en un proceso independiente. ¿Se habrían obtenido los mismos beneficios si, en cambio, Chrome se hubiera diseñado para abrir cada pestaña nueva en un hilo independiente? Explique.

4.13 ¿Es posible tener concurrencia pero no paralelismo? Explique.

4.14 Utilizando la Ley de Amdahl, calcule la ganancia de aceleración para la siguiente aplicación. cationes:

- 40 por ciento de paralelismo con (a) ocho núcleos de procesamiento y (b) dieciséis núcleos de procesamiento
- 67 por ciento de paralelismo con (a) dos núcleos de procesamiento y (b) cuatro núcleos de procesamiento

Machine Translated by Google



- 90 por ciento de paralelismo con (a) cuatro núcleos de procesamiento y (b) ocho núcleos de procesamiento

4.15 Determine si los siguientes problemas presentan paralelismo de tareas o de datos:

- Utilizar un hilo separado para generar una miniatura para cada foto en una recopilación
- Transposición de una matriz en paralelo •

Una aplicación en red donde un hilo lee desde la red y otro escribe a la red

- La aplicación de suma de matrices mediante bifurcación y unión descrita en la Sección

4.5.2 • El sistema de despacho de Grand Central

4.16 Un sistema con dos procesadores de doble núcleo tiene cuatro procesadores disponibles para la planificación. En este sistema se está ejecutando una aplicación que consume muchos recursos de la CPU. Toda la entrada de datos se realiza al inicio del programa, cuando se debe abrir un único archivo. De forma similar, toda la salida se realiza justo antes de que el programa finalice, cuando los resultados deben escribirse en un único archivo. Entre el inicio y la finalización, el programa depende completamente de la CPU. Su tarea consiste en mejorar el rendimiento de esta aplicación mediante el uso de multihilo.
La aplicación se ejecuta en un sistema que utiliza el modelo de subprocesos uno a uno (cada subproceso de usuario se corresponde con un subproceso del kernel).

- ¿ Cuántos hilos crearás para realizar la entrada y la salida? Explicar.
- ¿ Cuántos subprocesos creará para la parte que consume muchos recursos de la CPU? ¿La aplicación? Explícalo.

4.17 Considere el siguiente segmento de código:

```
pid t_pid;

pid = fork(); if (pid
== 0) { /* proceso hijo */ fork(); thread create( .

                    _              . .);

} tenedor();
```

a. ¿Cuántos procesos únicos se crean?

b. ¿Cuántos hilos únicos se crean?

4.18 Como se describe en la Sección 4.7.2, Linux no distingue entre procesos e hilos. En cambio, los trata de la misma manera, permitiendo que una tarea se asemeje más a un proceso o a un hilo según el conjunto de argumentos pasados a la llamada al sistema clone() . Sin embargo, otros sistemas operativos, como Windows, tratan los procesos e hilos de manera diferente. Por lo general, estos sistemas utilizan una notación en la que la estructura de datos de un proceso contiene punteros a los hilos individuales que pertenecen a dicho proceso. Compare estos dos enfoques para modelar procesos e hilos dentro del núcleo.

Machine Translated by Google



```
#include <pthread.h> #include
<stdio.h>

int valor = 0; void
*runner(void *param); /* el hilo */

int main(int argc, char *argv[]) { pid t pid; pthread t tid;

pthread attr t attr;

      pid = fork();

      if (pid == 0) { /* proceso hijo */ pthread attr init(&attr); pthread
            create(&tid,&attr,runner,NULL); pthread
            join(tid,NULL); printf("HIJO: valor = %d",value); /* LÍNEA C */

      }
      else if (pid > 0) { /* proceso padre */ wait(NULL); printf("PARENTE:
            valor = %d",valor); /
            * LÍNEA P */
      }
}

void *runner(void *param) { valor = 5; pthread
      exit(0);
}
```

Figura 4.22 Programa en C para el Ejercicio 4.19.

4.19 El programa que se muestra en la Figura 4.23 utiliza la API Pthreads. ¿Cuál sería la salida del programa en las líneas C y P?

4.20 Considere un sistema multinúcleo y un programa multihilo escrito utilizando el modelo de hilos de muchos a muchos. Suponga que el número de hilos a nivel de usuario en el programa es mayor que el número de núcleos de procesamiento en el sistema. Analice las implicaciones de rendimiento de lo siguiente: escenarios.

a. El número de subprocesos del kernel asignados al programa es menor que el número de núcleos de procesamiento.

b. El número de subprocesos del kernel asignados al programa es igual al número de núcleos de procesamiento.

Machine Translated by Google



```
estado antiguo;

pthread setcancelstate(PTHREAD CANCEL DISABLE, &oldstate);

/* ¿Qué operaciones se realizarían aquí? */

pthread setcancelstate(PTHREAD CANCEL ENABLE, &oldstate);
```

Figura 4.23 Programa en C para el Ejercicio 4.21.

c. El número de subprocesos del kernel asignados al programa es mayor que el número de núcleos de procesamiento pero menor que el número de subprocesos de nivel de usuario.

4.21 Pthreads proporciona una API para gestionar la cancelación de hilos. La función pthread setcancelstate() se utiliza para establecer el estado de cancelación. Su prototipo es el siguiente:

```
pthread setcancelstate(int estado, int *estadoantiguo)
```

Los dos valores posibles para el estado son PTHREAD CANCEL ENABLE y PTHREAD CANCEL DISABLE.

Utilizando el segmento de código que se muestra en la Figura 4.24, proporcione ejemplos de dos operaciones que serían adecuadas para realizar entre las llamadas para deshabilitar y habilitar la cancelación de subprocesos.